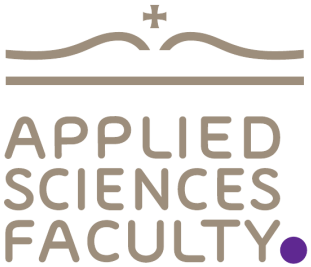
C++ — лекція 9. Простори імен





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Простори імен

Будь-яка нетривіальна програма складається з багатьох логічно пов'язаних, але незалежних частин, поміж яких — як елементи, створені її авторами, так і сторонні — різноманітні бібліотеки.

Простори імен – засіб реалізувати таке логічне групування.

Якщо клас можна собі уявити як конкретний прилад (що об'єднує в собі різноманітні деталі), то простір імен — свого роду поличка для приладів, які використовуються у вирішенні якоїсь конкретної задачі.

Простір імен є областю видимості. Оголошуються вони так: **my_lib.h**

```
namespace MyLib
{
    int fn1();
    void fn2(double);
}
```

Використання виглядатиме якось:

my_main.cpp <— так

```
#include "my_lib.h"
 ......
using MyLib::fn2;
int main(){
    ...........
    int x = MyLib::fn1();
    fn2(3.3);
    ...........
}
```

my_main.cpp <— або так

```
#include "my_lib.h"
 ......
using namespace MyLib;
int main(){
    ...........
    int x = fn1();
    fn2(3.3);
    ...........
}
```

my_lib.h <— Нехай маємо той же простір імен

```
namespace MyLib
{
    int fn1();
    void fn2(double);
}
```

Визначити оголошені в просторі імен сутності за його межами можна так:

my_lib.cpp

```
#include "my_lib.h"

int MyLib::fn1(){
 .....
}
void MyLib::fn2(double)
{
 .....
}
```

my_lib.cpp

```
#include "my_lib.h"

namespace MyLib
{
int fn1(){
 .....
}
void fn2(double)
{
 .....
```

- Простори імен можна розширювати:

```
namespace MyLib{
    int  fn1();
    void fn2(double);
}
    .....

namespace MyLib{
    int  fn3();
}
```

- Зокрема, коли включаються заголовки стандартної бібліотеки, кожен із них доповнює видимий в даній одиниці трансляції простір імен std.

- Оголошувати сутності з простору імен за його межами не можна:

  ```
  double MyLib::fn4(); // Помилка!
  ```

  але це не проблема, за потреби можна зробити так:

  ```
  namespace MyLib{
      double fn4();
  }
  ```

- Додавати щось своє до простору імен `std` – це невизначена поведінка.
- Невелика кількість винятків описана тут: `https://en.cppreference.com/w/cpp/language/extending_std`, наприклад – наприклад, підтримка Structured Bindings користувацьких типів).
- Див. також: `https://quuxplusone.github.io/blog/2021/08/06/dont-explicitly-instantiate-std-templates/`.

using-оголошення

- Простір імен є областю видимості, тому все, оголошене в ньому або в просторах видимості, що його охоплюють, видиме без спеціального уточнення[1]. В інших випадках простір імен потрібно вказати явно, за допомогою **кваліфікатора**:

```
MyLib::fn1();
```

- Якщо певне ім'я використовується часто, щоб постійно не вказувати кваліфікатор, можна "притягнути"ім'я в поточну область видимості за допомогою **using-оголошення**:

```
using MyLib::fn1();
  ......
fn1();
```

- C++17 дозволив передавати кілька імен через кому:

```
using MyLib::fn1, MyLib::fn2, OtherLib::fn3, std::cout;
```

[1] В тому числі і під час написання визначень оголошених в просторах імен сутностей.

using-директива

- Всі імена з простору імен можна зробити доступними за допомогою **using-директиви**:

  using namespace MyLib;

  Робити так, без крайньої потреби, не варто! using-директиви засмічують область видимості всіма іменами на купу, нейтралізуючи саму ідею просторів імен.
- Using namespace робить вигляд, що простір імен оголошено в найближчому просторі імен, спільному для поточного простору імен і того, який імпортується. Якщо таких немає – так, ніби вони оголошені глобально.
- Про відмінність між поведінкою using-директиви[2] і using-оголошенням: `https://quuxplusone.github.io/blog/2020/12/21/using-directive/`.

[2] Неочевидною – імена "притягуються", з точки зору поточної області видимості, не в неї, а в спільну точку поточного простору імен і того, яке using namespace – той простір імен (включаючи глобальні), що є спільним для них.

- Різні одиниці трансляції[3] можуть бачити різні оголошення простору імен. Зокрема, можна зробити так, щоб реалізація бібліотеки бачила більше імен, (які стосуються власне реалізації), ніж її клієнти, (яким потрібен тільки публічний інтерфейс).
- Найпростіше суть цього видно на прикладі програми із декількох файлів — кожен із них може включати різні заголовки стандартної бібліотеки, тому бачитиме різні частини простору імен std.
- Просторам імен можна оголошувати синоніми:

```
namespace Long_Named_Namespace {
    namespace Nested_But_Also_Long_Named_Namespace {
    }
}

namespace ShortName = Long_Named_Namespace;
namespace NestedShortName =
Long_Named_Namespace::Nested_But_Also_Long_Named_Namespace;
```

- В C++17 замість namespace A { namespace B { namespace C { ... } } } можна писати namespace A::B::C { ... }. C++20 дозволив у такому записі використовувати і inline простори.

[3] Якщо просто — різні .cpp файли.

- Існують анонімні (не-іменовані) простори імен. Імена, оголошені в них, видимі лише в даній одиниці трансляції.
- Вони використовуються для того ж, що і статичні (static) функції та статичні глобальні змінні — сутності, оголошені в таких просторах імен можна використовувати в даному файлі (даній одиниці трансляції), але лінкер не використовуватиме їх при пошуку визначень сутностей, оголошених в інших файлах:

```
namespace
{
    int x;
    void fn1(){}
}

int fnx()
{
    x=5;
    fn1();
}
```

- Фактично, створення анонімного простору імен можна собі уявити так:

```
namespace <Unique_name_unknown_to_programmer>
{
    ..............
}
using namespace <Unique_name_unknown_to_programmer>;
```

Тепер сутності, оголошені в ньому, ніхто ззовні використати не зможе, але в даному файлі (одиниці трансляції) вони доступні без якихось уточнень.

Простори імен можна об'єднувати:

```
namespace S1{
    int fn1();
    int fn1a();
}
namespace S2{
    int fn2();
}

namespace Merged
{
    using namespace S1;
    using namespace S2;
    int fn3();
}
```

Можна об'єднувати не тільки за допомогою using-директиви, але і using-оголошень:

```
namespace Merged2
{
    using S1::fn1();
    using S2::fn2();
}
```

Якщо в просторах імен, які об'єднуємо, є однакові імена, можна за допомогою using-оголошень вказати, котрі із них використовувати:

```
namespace P1{
    int fn1(int);
    int fn2(double);
}
namespace P1{
    int fn1(int);
    int fn1(double);
    int fn2(int);
}

namespace Merged{
    using namespace P1;
    using namespace P2;

    using P1::fn1;
    using P2::fn2;
}
```

Без using-оголошення, спроба викликати функцію Merged::fn1() даватиме помилку — неоднозначність перевантаження.
Імена, оголошені в даному просторі імен мають вищий пріоритет, ніж включені з інших.

Важливі особливості просторів імен

- Перевантаження імен діє скрізь простори імен. Тобто, якщо функції з однаковими іменами є в кількох видимих в той момент (тобто – доступних) просторах імен, вибір для перевантаження відбуватиметься між всіма ними.
- **Пошук імен**: так як функції для роботи із певними типами часто оголошені в тих же просторах імен, що й типи, якщо функція для аргументу певного типу не знайдена в поточній області видимості, її пошук здійснюється в просторі імен, де ці типи оголошено.
- Насправді, попереднім правилом ми постійно користуємося. Коли пишеться код:

```
std::cout << std::endl;
```

насправді, << це якась така функція: `ostream& std::operator<<(ostream&, ArgType)`. Однак, компілятор, подивившись, що аргументи із std, сам її знаходить, і ми не мусимо писати якийсь такий жах:

```
std::operator<<(std::cout, std::endl);
```

- `https://quuxplusone.github.io/blog/2021/10/27/dont-reopen-namespace-std/` — чому не можна додавати щось до std.

inline namespace (C++11)

```
namespace A {
    inline  namespace B {
        ....
    }
     ......
}
```

- Імена, оголошені в inline просторі імен видимі в просторі імен, в якому їх оголошено – використовується неявна директива using namespace.
- Inline – властивість транзитивна, вкладені inline-простори імен належать найближчому не-inline.
- Анонімні простори імен можуть бути inline.
- Використання – можливість одночасного існування кількох версій бібліотеки, з одним, вибраним за замовчуванням. Приклад – див. далі.
- Також, std::string_literals і std::chrono_literals реалізовані як inline-простори імен в std::literals. Завдяки цьому, їх можна підключати як всіх разом, так і частинами.

Inline-простори імен для версіонування бібліотек

```
namespace SomeLibrary
{
    #ifdef SOME_LIBRARY_USE_VERSION_2
        inline namespace version_2 { }
    #else
        inline namespace version_1 { }
    #endif
    namespace version_1 {
        template <typename T> int foo(T a) {return 1;}
    }
    namespace version_2 {
        template <typename T> int foo(T a) {return 2;}
    }
}
```

- Детальніше див. `https://en.cppreference.com/w/cpp/language/namespace`.

Атрибути просторів імен (C++17)

- Простори імен, включаючи inline, можуть мати атрибути, що будуть стосуватися всіх сутностей, в них оголошених:

```
namespace [[deprecated]] original_lib {
    void fn4();
}
```

Посилання

- ”True names matter in C++”



